Manejo de Datos

Facultad de Ciencias, UNAM

Gerardo Avilés Rosas gar@ciencias.unam.mx

Tarea 4	**TAD Lista y TAD Pila**
Fecha de entrega	**30 octubre de 2026**

1. Directorio Escolar con Listas

Para esta tarea vas a **retomar el problema** que resolviste en la tarea de **Repaso de POO**, y deberás de hacer una **actualización** a tu solución. La **actualización** consiste en **quitar la limitante** que tiene el **directorio**, la cual **obliga** a **decidir** desde un **inicio** el **número máximo de contactos** a guardar (se hizo un ajuste para lograr **"crecer"** el arreglo, pero ***la solución no es eficiente***).

Se pide entonces que **cambies la estructura de almacenamiento de datos**, por una **lista** (debes utilizar la versión de ***lista simplemente ligada con centinela al inicio***), para lograr tener una **organización completamente dinámica**. Dado que **forma de administrar** los datos **cambia**, **varios métodos** que actualmente están implementados en el directorio **también deberán cambiar**. En este sentido, deberás indicar a través de una **tabla cuáles** son los **métodos** que tienes **implementados** con la **solución de arreglos**, **cuáles** de ellos **se mantienen** en la **implementación con listas**. Para aquellos **métodos que se mantienen**, deberás **indicar si no existe cambio** en su forma de trabajo o **si se debe ajustar** la programación, en cuyo caso, **deberás indicar el cambio que realizaste**.

El **segundo cambio** que se debe agregar es lo que tiene que ver con la **persistencia de los datos** que se manejan. En esta nueva versión, **se debe garantizar** que el programa **permita** (si así lo desea el usuario) **cargar datos** que se tengan **previamente** del directorio. Los **datos deberán leerse** desde un **archivo** en **formato CSV** (*separado por comas*). En el caso de que **el usuario disponga de este archivo**, el programa **deberá preguntar** previamente al usuario **cómo es que desea comenzar el trabajo** del directorio: un **directorio en blanco** (*sin ningún contacto registrado*) o bien, con los **datos contenidos en un archivo**; en este caso, el programa **deberá solicitar el nombre del archivo** que se desea leer.

Debido a que **durante la ejecución** del programa **es posible** que los **datos puedan cambiar** (*actualizar algún contacto, agregar nuevos contactos o eliminar contactos almacenados*), una vez que el **usuario decida finalizar la ejecución**, el programa **deberá automáticamente guardar los cambios** en el **archivo leído**. Si el programa **inició** con un **directorio en blanco**, se **deberá solicitar** al usuario el **nombre del archivo** donde se van a **guardar los datos** (*en formato CSV*). En caso de que el usuario haya **previamente cargado** los datos **desde un archivo**, este será el **archivo que se actualizará** con los cambios realizados.

Para **las opciones** en las que se pida **mostrar la información ordenada**, se **deberá de utilizar** la implementación de **listas ordenadas** que se proporcionó en el material, para llevar a cabo el proceso.

Es importante recordar que el **recorrido de la lista** se hace a través del **iterador**, así que deberás apoyarte en esta implementación cuando **requieras acceder** a los contactos almacenados. No debes olvidar el tema de la **documentación completa** de tus clases, la cual se debe entregar en formato **docString**.

2. Pilas de Libros

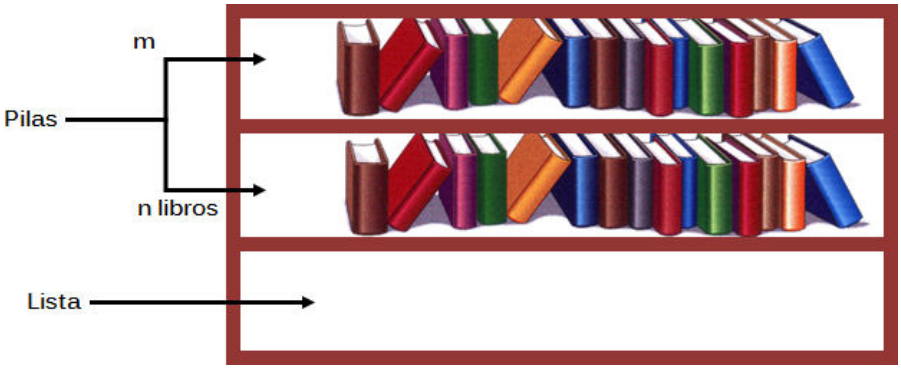
Un **bibliotecario** tiene el siguiente problema: tiene **un estante** con **tres repisas** en el cual en **dos de ellas** existen m y n **libros respectivamente**, de **autores y géneros varios**. Considera que los **Libros** tienen un **título**, **autor**, **editorial** y **año** de publicación.

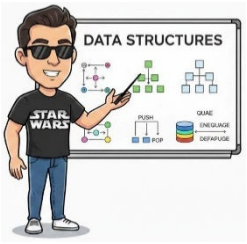
Escribe un programa que **ordene los Libros** en la **tercera repisa restante**, con la **restricción** de que la **única forma de sacar los libros** de las **dos repisas** donde se **encuentran originalmente** con la **política** $LIFO$ y la **repisa en la cual deben estar ordenados** la puedes modelar como una **Lista Ordenada**.

Considera que los **Libros** deben ser tratados **como objetos** y que en la **tercera repisa** el **bibliotecario** puede **decidir ordenar** los libros de acuerdo con **tres posibles criterios**: **por título**, **por autor** y **por editorial**.

Los **Libros** que se van a utilizar para **llenar** los primeros **dos estantes**, los encuentras en los **archivos** $estante1.csv$ y $estante2.csv$, por lo que **deberás asegurarte** que tu programa pueda **leer los datos** desde **archivos CSV**. Al momento de solicitar **ordenar los libros**, se **debe preguntar al usuario** sobre **qué criterio de ordenamiento** se realizará el **acomodo de los Libros** (deberás de **dar posibilidad** de **tres criterios** al menos). Finalmente, el programa **deberá permitir** que se pueda guardar en **un tercer archivo**, la lista de **Libros ordenados**.

Es importante recordar que el **recorrido de la** $Pila$ se hace a través del $iterador$, así que puedes apoyarte en esta **implementación** cuando **requieras acceder** a los libros. No debes olvidar el tema de la **documentación completa** de tus clases, la cual se debe entregar en formato $docString$.





Deberán subir su tarea a **Google Classroom**, de acuerdo lo indicado en los **lineamientos de entrega** de las **tareas**. La tarea se entrega de acuerdo con los **equipos que hayan formado**.

Nota: Para cualquier **duda** o **comentari**o que pudiera surgirles al hacer su tarea, recuerden que cuentan con el **foro de dudas de la tarea** en **Classroom**. Pueden también **dirigir sus dudas** al grupo de **Telegram**.